

Polynomial Operations in C

```
#include <stdio.h>
#include <stdlib.h>

// Define structure for a polynomial term
struct Term
{
    float coeff;           // Coefficient of the term
    int expo;              // Exponent of the term
    struct Term *nextTerm; // Pointer to the next term
};

// Function to insert a term into the polynomial in sorted
// order
// Handles duplicate exponents by summing coefficients
struct Term *insert(struct Term *head, struct Term
*newTerm)
{
    struct Term *temp = head;
    struct Term *prev = NULL;

    // Case 1: Empty list
    if (head == NULL)
    {
        return newTerm;
    }

    // Traverse the list to find the correct position or
    // handle duplicates
    while (temp != NULL && temp->expo >= newTerm->expo)
    {
        if (temp->expo == newTerm->expo)
        {
            // Duplicate found → add coefficients
            temp->coeff += newTerm->coeff;
            free(newTerm); // Free unused node
            return head;
        }
    }
}
```

```

        }
        prev = temp;
        temp = temp->nextTerm;
    }

    // Insert at the correct position
    newTerm->nextTerm = temp;
    if (prev == NULL) // Insert at head
        head = newTerm;
    else
        prev->nextTerm = newTerm;

    return head;
}

// Create a new term node
struct Term *createTerm(float coeff, int expo)
{
    struct Term *newTerm = (struct Term
*)malloc(sizeof(struct Term));
    newTerm->coeff = coeff;
    newTerm->expo = expo;
    newTerm->nextTerm = NULL;
    return newTerm;
}

// Print the polynomial in human-readable format
void print(struct Term *head)
{
    if (head == NULL)
    {
        printf("Polynomial is empty.\n");
        return;
    }

    while (head != NULL)
    {
        printf("%.1fx^%d", head->coeff, head->expo);
        head = head->nextTerm;
        if (head != NULL)
            printf(" + ");
    }
}

```

```

    }
    printf("\n");
}

// Free all memory allocated for a polynomial
void cleanup_poly(struct Term **head)
{
    struct Term *temp;
    while (*head != NULL)
    {
        temp = *head;
        *head = (*head)->nextTerm;
        free(temp);
    }
}

// Add two polynomials and return the result list
struct Term *add(struct Term *head1, struct Term *head2,
struct Term *head3)
{
    cleanup_poly(&head3);
    struct Term *newTerm;

    while (head1 != NULL && head2 != NULL)
    {
        if (head1->expo == head2->expo)
        {
            newTerm = createTerm(head1->coeff + head2-
>coeff, head1->expo);
            head3 = insert(head3, newTerm);
            head1 = head1->nextTerm;
            head2 = head2->nextTerm;
        }
        else if (head1->expo > head2->expo)
        {
            newTerm = createTerm(head1->coeff, head1-
>expo);
            head3 = insert(head3, newTerm);
            head1 = head1->nextTerm;
        }
        else

```

```

        {
            newTerm = createTerm(head2->coeff, head2-
>expo);
            head3 = insert(head3, newTerm);
            head2 = head2->nextTerm;
        }
    }

    while (head1 != NULL)
    {
        newTerm = createTerm(head1->coeff, head1->expo);
        head3 = insert(head3, newTerm);
        head1 = head1->nextTerm;
    }

    while (head2 != NULL)
    {
        newTerm = createTerm(head2->coeff, head2->expo);
        head3 = insert(head3, newTerm);
        head2 = head2->nextTerm;
    }

    return head3;
}

// Subtract two polynomials and return the result list
struct Term *sub(struct Term *head1, struct Term *head2,
struct Term *head3)
{
    cleanup_poly(&head3);
    struct Term *newTerm;

    while (head1 != NULL && head2 != NULL)
    {
        if (head1->expo == head2->expo)
        {
            newTerm = createTerm(head1->coeff - head2-
>coeff, head1->expo);
            head3 = insert(head3, newTerm);
            head1 = head1->nextTerm;
            head2 = head2->nextTerm;
        }
    }
}

```

```

        }
        else if (head1->expo > head2->expo)
        {
            newTerm = createTerm(head1->coeff, head1-
>expo);
            head3 = insert(head3, newTerm);
            head1 = head1->nextTerm;
        }
        else
        {
            newTerm = createTerm(-head2->coeff, head2-
>expo);
            head3 = insert(head3, newTerm);
            head2 = head2->nextTerm;
        }
    }

    while (head1 != NULL)
    {
        newTerm = createTerm(head1->coeff, head1->expo);
        head3 = insert(head3, newTerm);
        head1 = head1->nextTerm;
    }

    while (head2 != NULL)
    {
        newTerm = createTerm(-head2->coeff, head2->expo);
        head3 = insert(head3, newTerm);
        head2 = head2->nextTerm;
    }

    return head3;
}

// Create polynomial by user input
struct Term *create(struct Term *head)
{
    int termNo, expo;
    float coeff;

    printf("\nEnter number of terms: ");

```

```

scanf("%d", &termNo);

for (int i = 0; i < termNo; i++)
{
    printf("\nTerm %d:\n", i + 1);
    printf("Enter Coefficient: ");
    scanf("%f", &coeff);
    printf("Enter Exponent: ");
    scanf("%d", &expo);

    struct Term *newTerm = createTerm(coeff, expo);
    head = insert(head, newTerm);
}

return head;
}

// Main function with menu-driven interface
int main()
{
    int choice;
    struct Term *head1 = NULL, *head2 = NULL, *head3 =
NULL;

    do
    {
        printf("\n===== \n");
        printf("1. Create Polynomials\n");
        printf("2. Add Polynomials\n");
        printf("3. Subtract Polynomials\n");
        printf("4. Print Polynomials\n");
        printf("5. Exit\n");
        printf("===== \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("\n--- Create First Polynomial ---\n");
                head1 = create(head1);

```

```

        printf("\n--- Create Second Polynomial ---\n");
        head2 = create(head2);
        break;

    case 2:
        printf("\n--- Addition Result ---\n");
        head3 = add(head1, head2, head3);
        print(head3);
        break;

    case 3:
        printf("\n--- Subtraction Result ---\n");
        head3 = sub(head1, head2, head3);
        print(head3);
        break;

    case 4:
        printf("\n--- Polynomial 1 ---\n");
        print(head1);
        printf("\n--- Polynomial 2 ---\n");
        print(head2);
        break;

    case 5:
        printf("\nExiting...\n");
        break;

    default:
        printf("\nInvalid choice. Try again.\n");
    }

} while (choice != 5);

cleanup_poly(&head1);
cleanup_poly(&head2);
cleanup_poly(&head3);

return 0;
}

```

Output :

```
=====
1. Create Polynomials
2. Add Polynomials
3. Subtract Polynomials
4. Print Polynomials
5. Exit
=====
Enter your choice: 1

--- Create First Polynomial ---

Enter number of terms: 4

Term 1:
Enter Coefficient: 6
Enter Exponent: 4

Term 2:
Enter Coefficient: 5
Enter Exponent: 5

Term 3:
Enter Coefficient: 3
Enter Exponent: 2

Term 4:
Enter Coefficient: 2
Enter Exponent: 3
```

```
--- Create Second Polynomial ---
```

```
Enter number of terms: 3
```

```
Term 1:
```

```
Enter Coefficient: 6
```

```
Enter Exponent: 5
```

```
Term 2:
```

```
Enter Coefficient: 4
```

```
Enter Exponent: 2
```

```
Term 3:
```

```
Enter Coefficient: 3
```

```
Enter Exponent: 2
```

```
=====
1. Create Polynomials
2. Add Polynomials
3. Subtract Polynomials
4. Print Polynomials
5. Exit
=====
```

```
Enter your choice: 4
```

```
--- Polynomial 1 ---
```

```
5.0x^5 + 6.0x^4 + 2.0x^3 + 3.0x^2
```

```
--- Polynomial 2 ---
```

```
6.0x^5 + 7.0x^2
```



```
=====
1. Create Polynomials
2. Add Polynomials
3. Subtract Polynomials
4. Print Polynomials
5. Exit
=====
Enter your choice: 2

--- Addition Result ---
11.0x^5 + 6.0x^4 + 2.0x^3 + 10.0x^2
```

```
--- Polynomial 1 ---
5.0x^5 + 6.0x^4 + 2.0x^3 + 3.0x^2

--- Polynomial 2 ---
6.0x^5 + 7.0x^2

=====
1. Create Polynomials
2. Add Polynomials
3. Subtract Polynomials
4. Print Polynomials
5. Exit
=====
Enter your choice: 3

--- Subtraction Result ---
-1.0x^5 + 6.0x^4 + 2.0x^3 + -4.0x^2

=====
1. Create Polynomials
2. Add Polynomials
3. Subtract Polynomials
4. Print Polynomials
5. Exit
```